AGENDA TELEFÔNICA

API REST, SQLite e aplicação web

Documentação técnica e guia de uso • 9 de novembro de 2026

Uma aplicação para manter contatos, grupos e favoritos em um único lugar. O projeto inclui interface em português, serviço HTTP, persistência real em banco SQL, testes e código-fonte executável sem dependências npm.

Entrega	Conteúdo
Aplicação	Cadastro, consulta, edição, exclusão, busca e favoritos.
Banco local	Arquivo agenda.sqlite, acessado pelo módulo node:sqlite.
Banco hospedado	Cloudflare D1, com semântica SQL baseada em SQLite.
Documentação	Arquitetura, modelo de dados, API, nuvem, execução e evidências.
GitHub	Publicação pendente de definição da conta/repositório de destino.

Escopo e evidências

As capturas deste documento foram produzidas na prévia local com um contato fictício. O teste automatizado usa SQLite real. A confirmação da publicação hospedada é informada separadamente na entrega; a existência de uma URL prevista, por si só, não comprova implantação.

01 / Usar a agenda

Clique em Novo contato, preencha nome e telefone e escolha um grupo. E-mail, empresa e anotações são opcionais na interface. Marque a opção de favorito para destacar pessoas frequentes. Ao salvar, a API valida os campos e grava o contato no banco.

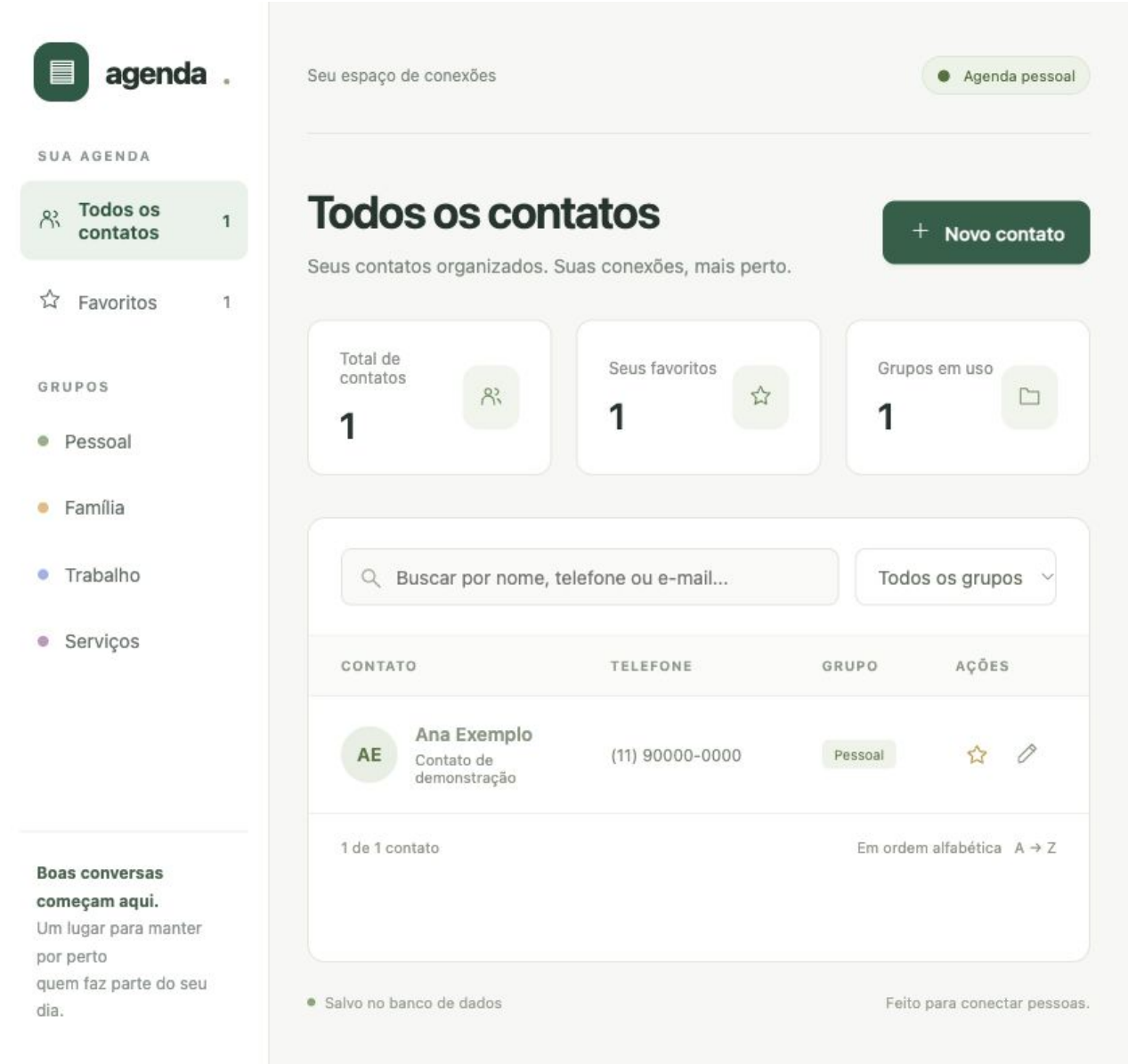


Figura 1. Lista na prévia local, com dados fictícios. Os cartões mostram totais, favoritos e grupos em uso. A busca aceita nome, telefone, e-mail e empresa; o filtro lateral restringe o grupo ou favoritos.

02 / Editar e consultar

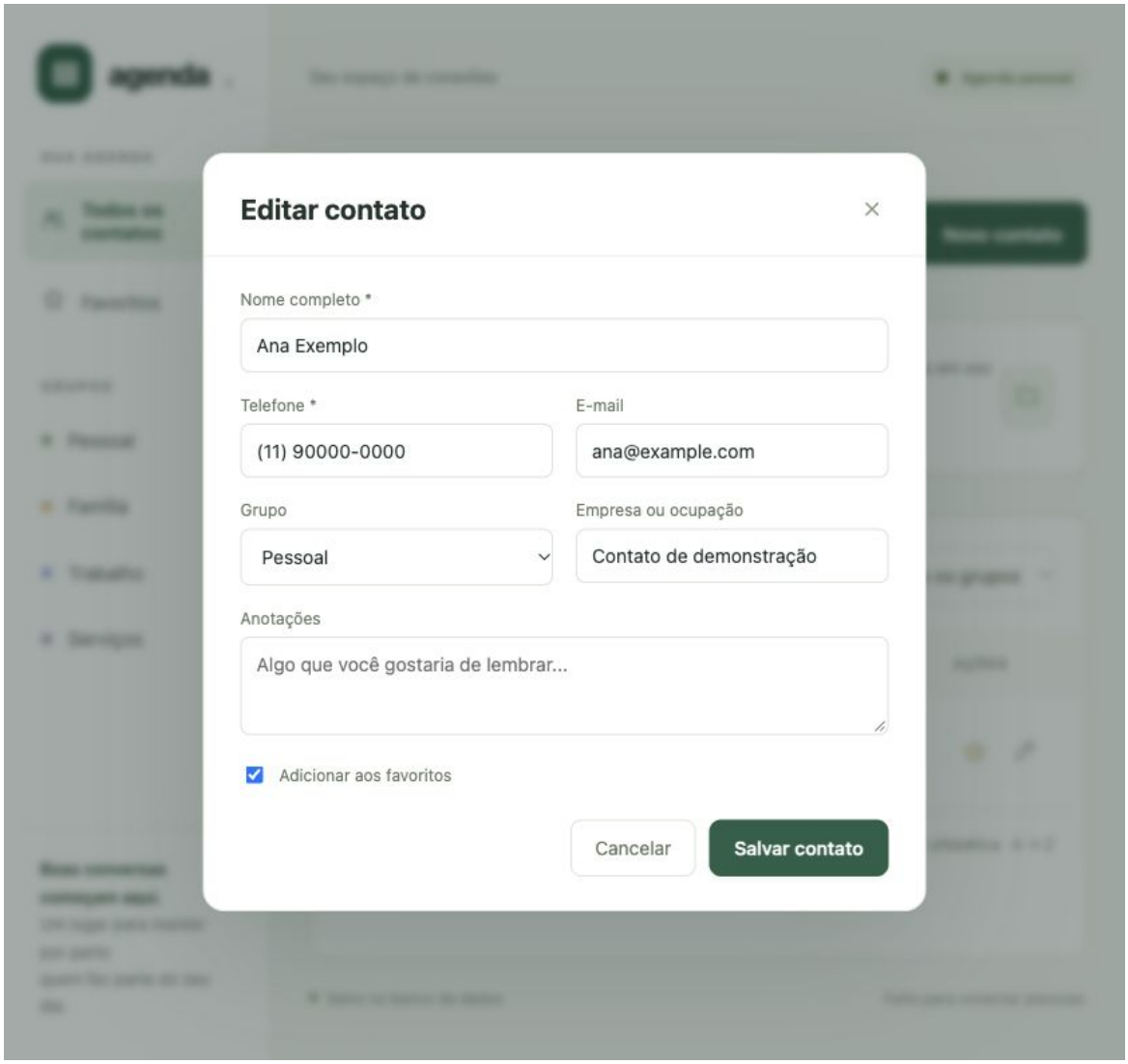


Figura 2. Formulário de edição. O botão de lápis abre os dados atuais. Clique no nome para ver detalhes e acessar a exclusão; uma confirmação é exigida antes da remoção. A estrela adiciona ou remove o favorito.

O nome deve ter ao menos 2 caracteres e o telefone de 8 a 15 dígitos. O e-mail é validado quando informado. Mensagens de erro aparecem na interface; a caixa de diálogo mantém os campos para correção.

03 / Arquitetura e persistência

Camada	Responsabilidade
Navegador	HTML, CSS e JavaScript; formulário e renderização da lista.
HTTP / JSON	Chamadas de mesma origem para /api/contacts.
Cloudflare Worker	Serve a interface, valida requisições e executa consultas SQL.
D1 (nuvem)	Persistência relacional acessada pelo binding DB.
Node.js (local)	Executa o mesmo handler com adaptador SQLite e servidor HTTP.

Fluxo visual

NAVEGADOR | Interface da agenda

↓ ***HTTPS e JSON***

WORKER | API REST e validação

↓ ***Consultas SQL parametrizadas***

D1 | Tabela contacts

No computador: Navegador → Node.js → agenda.sqlite. Os bancos local e hospedado são independentes e não sincronizam automaticamente. Os contatos não são salvos em localStorage.

Uma única tabela atende à agenda pequena. As categorias são valores fixos; não há tabela de grupos nem relacionamento entre vários usuários. O índice de nome apoia a ordenação, mas a busca textual atual ocorre após carregar os registros.

04 / Modelo de dados

Campo	Tipo	Uso
id	TEXT / PK	UUID gerado no servidor.
name	TEXT	Nome, até 100 caracteres.
phone	TEXT	Telefone formatado, até 30 caracteres.
email	TEXT	E-mail ou string vazia, até 150.
category	TEXT	Pessoal, Família, Trabalho ou Serviços.
company	TEXT	Empresa/ocupação, até 100.
notes	TEXT	Anotações, até 1.000.
favorite	INTEGER	0 ou 1; o JSON de escrita usa booleano.
created_at	TEXT	Instante de criação ISO 8601.

A migration está em drizzle/0000_contacts.sql. Na hospedagem, o fluxo de publicação aplica as migrations antes de enviar o Worker. No servidor local, a inicialização cria a tabela se ela ainda não existir.

Consultas usam placeholders e valores vinculados, sem interpolar dados do contato no SQL. A interface escapa texto antes de exibi-lo, e o servidor limita campos e tamanho das requisições. A integridade de nomes e telefones também depende da validação da API.

05 / Contrato da API

Método / caminho	Resultado
GET /api/health	200: banco responde; 503: indisponível.
GET /api/contacts?q=ana	200: lista; busca por texto opcional.
GET /api/contacts/:id	200: um contato; 404: ausente.
POST /api/contacts	201: contato criado.
PUT /api/contacts/:id	200: contato atualizado; corpo completo.
DELETE /api/contacts/:id	204: excluído, sem corpo.

Exemplo de corpo para POST ou PUT

```
{
  "name": "Ana Exemplo",
  "phone": "(11) 90000-0000",
  "email": "ana@example.com",
  "category": "Pessoal",
  "company": "",
  "notes": "",
  "favorite": false
}
```

Use Content-Type: application/json. Os sete campos de texto e favorite precisam estar presentes no corpo. A resposta acrescenta id e created_at; favorite é devolvido como 0/1.

Erros de validação/JSON retornam 400; origem externa, 403; recurso ausente, 404; método inválido, 405; corpo grande, 413; tipo de conteúdo, 415; falha interna, 500. O corpo de erro contém a chave error.

A busca na interface ignora acentos e ordena em português. A busca da API usa minúsculas sem normalização de acentos, e a ordenação SQL NOCASE pode diferir da interface.

06 / Nuvem, execução e entrega

Hospedagem

O Sites administra a URL HTTPS, a publicação privada e o Worker. O D1 mantém os dados SQL. Não foi contratada uma máquina virtual: SKU de VM, disco de VM e sistema operacional administrado pelo aluno não se aplicam. O projeto não utiliza R2. Plano comercial, custo, quotas da conta e região efetiva não foram informados pela plataforma. Os Workers têm limite documentado de 128 MB por isolate.

O acesso permanece privado ao proprietário pela plataforma. Não há autenticação própria na API nem isolamento por usuário. Se o acesso for ampliado no futuro, os visitantes autorizados verão a mesma agenda; para contas independentes, será necessário adicionar owner_id e autorizar todas as consultas.

Execução local

Instale Node.js 22.13+; o projeto foi testado em 24.22.1. Descompacte o código e execute os comandos na pasta do projeto. Nenhum npm install é necessário.

```
node scripts/local.mjs
# Abra http://127.0.0.1:5173
node --test scripts/api.test.mjs
node scripts/build.mjs
```

O SQLite é criado no diretório de trabalho. Para backup local, pare o serviço e copie agenda.sqlite. O servidor escuta em 127.0.0.1. A cópia de código entregue não inclui contatos ou credenciais.

GitHub e publicação

O Git de hospedagem Sites não é um repositório GitHub. Para concluir essa parte da referência, informe o repositório desejado. O README inclui comandos para publicar em um repositório vazio criado pelo usuário. Não foi inventado um link de GitHub.

07 / Verificação e limites

Verificação	Evidência
Teste automatizado	CRUD; busca; favorito; telefone e JSON inválidos; rejeição de origem externa; registro ausente.
Persistência	Banco fechado e reaberto; contato e favorito mantidos.
Interface	Cadastro e abertura do formulário de edição conferidos no navegador.
Build	Worker ESM gerado; verificação sintática executada.
Capturas	Prévia local com contato fictício; não são prints de dados em produção.

Limites conhecidos

A versão se destina a uma agenda pequena. Não há paginação, importação/exportação, fotos, auditoria, backup pela interface ou teste de carga. A API lê toda a lista antes de filtrar. A exclusão é definitiva na aplicação. O ambiente local não reproduz integralmente autenticação, infraestrutura e desempenho da nuvem.

Referências oficiais

Cloudflare D1: developers.cloudflare.com/d1/

Limites Workers: developers.cloudflare.com/workers/platform/limits/

SQLite no Node.js 24: nodejs.org/docs/latest-v24.x/api/sqlite.html

Fontes consultadas em 9 de novembro de 2026. Consulte a documentação e o plano da conta para limites e preços vigentes.

Arquivos para estudar

worker/ui.js: interface. worker/index.js: rotas e validação. scripts/local.mjs: servidor e adaptador SQLite. scripts/api.test.mjs: testes. scripts/build.mjs: build. drizzle/: schema SQL. README.md: instruções e exemplos completos.